

0、问题探查模板

2021年12月20日 14:03

--- (1) 查看IDE配置

```
select job_name, responsible_user_name
from bi_ssa.tssa_ide_job t
where t.statis_date='20211129' --and system_id='12577'--系统为fbi
and
(
    upper(job_config) like upper('%UBA_PIT_CONTENT_DETAIL%')
)
```

1、sql 中 \${} 和 #{} 的区别

2021年10月29日 16:48

项目开发过程中，在mybatis框架中经常sql需要动态赋值，会出现#{param}、\${param}两种形式。

接下来，我们一起来看看一个案例：根据用户的姓名来筛选用户信息，其中用户姓名不确定，是动态变化的，sql如下：

select * from userInfo where user_name= "张三" ; //查询名称是张三的信息

在xml中， select * from userInfo where user_name=#{name} //根据名称动态查询用户信息, #将参数解析成字符串

或者：select * from userInfo where user_name= '\${name}' //\$会将参数解析成对象 需要加上引号引住

一、主要区别如下：

- 1、#将传入的参数都当成一个字符串，会对自动传入的数据加一个双引号。如：order by #{age}，如果传入的值是18,那么解析成sql时的值为order by "18", 如果传入 age ,则会解析为 order by "age"
- 2、\$将传入的参数直接显示生成在sql中,被当成一个对象。如：order by \${age}，如果传入的值是18,那么解析成sql时的值为order by 18, 如果传入的值是age，则解析成的sql为order by age
- 3、#方式底层采用预编译方式PreparedStatement，能够很大程度防止sql注入；\$方式底层只是Statement，无法防止Sql注入。
- 4、\$方式一般用于传入数据库对象，例如传入表名。
- 5、一般能用#的就别用\$ 注意点：MyBatis排序时使用order by 动态参数时需要注意，用\$而不是#

2、Case....when.....

2021年11月1日 17:58

Case具有两种格式。简单Case函数和Case搜索函数。

简单Case函数

```
CASE sex  
WHEN '1' THEN '男'  
WHEN '2' THEN '女'  
ELSE '其他' END
```

--Case搜索函数

```
CASE WHEN sex = '1' THEN '男'  
WHEN sex = '2' THEN '女'
```

两种方式都可以实现相同的功能。

```
select  
    case when is_first_prch_acct='N' then '不是首购' else '首购' end as first_prch_acct  
    ,count(distinct acct_no)  
from fdm_dpa.fin_bof_trade_size_list a  
where stat_date='20211015'  
group by case when is_first_prch_acct='N' then '不是首购' else '首购' end
```

3、统计当周、当月、双周

2021年11月1日 19:53

1、统计一周内数据

```
stat_date>=regexp_replace(date_sub(from_unixtime(unix_timestamp('${statisdate}','yyyyMMdd'),'yyyy-MM-dd'),pmod(datediff(from_unixtime(unix_timestamp('${statisdate}','yyyyMMdd'),'yyyy-MM-dd'),'1900-01-08'),7)),'-','') ---定位到当周周一  
and stat_date<='${statisdate}'
```

pmod(int a, int b)

pmod(double a, double b)

返回a除b的余数的绝对值。

当 b为7

输出的结果为0-6的数，分别表示 日，一，二 ... 六。

2、统计当月数据

```
where substr(stat_date,1,6)=substr('${statisdate}',1,6)  
and stat_date<='${statisdate}'
```

4、regexp_replace函数

2021年11月1日 20:05

命令格式:

regexp_replace(source, pattern, replace_string, occurrence)

参数说明:

- source: string类型, 要替换的原始字符串。
- pattern: string类型常量, 要匹配的正则模式, pattern为空串时抛异常。
- replace_string:string, 将匹配的pattern替换成的字符串。
- occurrence: bigint类型常量, 必须大于等于0,

大于0: 表示将第几次匹配替换成replace_string,

等于0: 表示替换掉所有的匹配子串。

其它类型或小于0抛异常。

返回值:

将source字符串中匹配pattern的子串替换成指定字符串后返回, 当输入source, pattern, occurrence参数为NULL时返回NULL, 若replace_string为NULL且pattern有匹配, 返回NULL, replace_string为NULL但pattern不匹配, 则返回原串。

常用案例

1、用' #' 替换字符串中的所有数字

```
SELECT regexp_replace('01234abcde56789','[0-9]','#') AS new_str FROM dual;
```

结果: #####abcde#####

用' #' 替换字符串中的数字0、9

```
SELECT regexp_replace('01234abcde56789','[09]',' #') AS new_str FROM dual;
```

结果: #1234abcde5678#

2、遇到非小写字母或者数字跳过, 从匹配到的第4个值开始替换, 替换为"

```
SELECT regexp_replace('abcdefg123456ABC','[a-z0-9]',",",4)
```

结果: abcefg123456ABC

```
SELECT regexp_replace('abcDEfg123456ABC','[a-z0-9]',",",4)
```

结果: abcDEg123456ABC

```
SELECT regexp_replace('abcDEfg123456ABC','[a-z0-9]',",",7);
```

结果: abcDEfg13456ABC

遇到非小写字母或者数字跳过, 将所有匹配到的值替换为"

```
SELECT regexp_replace('abcDefg123456ABC','[a-z0-9]',",",0);
```

结果: DABC

3、格式化手机号, 将+86 13811112222转换为(+86) 138-1111-2222, ' + '在正则表达式中有定义, 需要转义。\\1表示引用的第一个组

```
SELECT regexp_replace('+86 13811112222','(\\+[0-9]{2})\\([0-9]{3}\\)([0-9]{4})\\([0-9]{4}\\)','(\\1)\\3-\\4-\\5',0);
```

结果: (+86)138-1111-2222

```
SELECT regexp_replace('123.456.7890','([[:digit:]]{3})\\.[[:digit:]]{3}\\.[[:digit:]]{4}','(\\1)\\2-\\3',0);
```

```
SELECT regexp_replace('123.456.7890','([0-9]{3})\\.[0-9]{3}\\.[0-9]{4}','(\\1)\\2-\\3',0);
```

结果: (123)456-7890

4、将字符用空格分隔开, 0表示替换掉所有的匹配子串。

```
SELECT regexp_replace('abcdefg123456ABC','(.)','\\1 ',0) AS new_str FROM dual;
```

结果: a b c d e f g 1 2 3 4 5 6 A B C

```
SELECT regexp_replace('abcdefg123456ABC','(.)','\\1 ',2) AS new_str FROM dual;
```

结果: ab cdefg123456ABC

5、

```
SELECT regexp_replace("abcd","(.*)($)","\\1",0);
```

结果: abc

```
SELECT regexp_replace("abcd","(.*)($)","\\2",0);
```

结果: d

```
SELECT regexp_replace("abcd","(.*)($)","\\1-\\2",0);
```

结果: abc-d

其他案例:

SELECT regexp_replace("abcd","(.)","\\2",1) 结果为"abcd", 因为pattern中只定义了一个组, 引用的第二个组不存在。

SELECT regexp_replace("abcd","(.*)(\$)","\\2",0) 结果为"d"

SELECT regexp_replace("abcd","(.*)(\$)","\\1",0) 结果为"abc"

SELECT regexp_replace("abcd","(.*)(\$)","\\1-\\2",0) 结果为"abc-d"

SELECT regexp_replace("abcd","a","\\1",0), 结果为" \\1bcd", 因为在pattern中没有组的定义, 所以\\1直接输出为字符。

正则符号释义:

[SparkSQL] regexp_replace函数使用 去除特殊隐藏字符\\n\\t\\r

1、函数介绍

REGEXP_REPLACE(inputString, regexString, replacementString)

第一个参数: 表中字段

第二个参数: 正则表达式

第三个参数: 要替换成为的字符

2、使用中的坑

函数使用起来比较简单, 但是也有坑, 当要匹配特殊的隐藏字符\\n \\r \\t,等回车符、制表符时, 需要通过使用四个\\ 进行转译。

符号	释义
\	代表它自己、引用下一个字符、引入一个操作符、什么也不做
*	匹配零或者多个
+	匹配一个或者多个
?	匹配零或者一个
	或运算，其左右操作数均可以作为一个子表达式
\$	默认情况下匹配字符串的结尾。在多行模式下，它匹配源字符串中任意位置的行尾
^	默认情况下匹配字符串的开头。在多行模式下，它匹配源字符串中任意位置的行头
.	匹配字符集中支持的任意字符，null除外
[]	用于指定匹配列表的括号表达式
()	对表达式进行分组，将其视为单个子表达式
[a]	恰好匹配a次
[a,]	至少匹配a次
[a,n]	匹配至少a次，但不超过n次
\n	反向引用表达式（n为1-9）匹配在\n之前的圆括号内包含的第n个子表达式
[...]	指定排序规则，可以是多字符元素（例如，西班牙语中的[.ch.]）
[::]	指定字符类（例如，[:alpha:]，它匹配字符类中的任何字符
[==]	指定等价类（例如，[a=]匹配索引具有基本字母a的字符

特殊符号	代表意义
[[:alnum:]]	代表英文大小写字符及数字，亦即 0-9, A-Z, a-z
[[:alpha:]]	代表任何英文大小写字符，亦即 A-Z, a-z
[[:blank:]]	代表空白键与 [Tab] 按键两者
[[:cntrl:]]	代表键盘上面的控制按键，亦即包括 CR, LF, Tab, Del.. 等等
[[:digit:]]	代表数字而已，亦即 0-9
[[:graph:]]	除了空白字符（空白键与 [Tab] 按键）外的其他所有按键
[[:lower:]]	代表小写字符，亦即 a-z
[[:print:]]	代表任何可以被打印出来的字符
[[:punct:]]	代表标点符号（punctuation symbol），亦即： " ' ? ! ; : # \$...
[[:upper:]]	代表大写字符，亦即 A-Z
[[:space:]]	任何会产生空白的字符，包括空白键, [Tab], CR 等等
[[:xdigit:]]	代表 16 进位的数字类型，因此包括： 0-9, A-F, a-f 的数字与字符

5、Spark-Sql内置函数日期处理

2021年11月1日 20:08

一、获取当前时间

1.current_date 获取当前日期

2018-04-09

2.current_timestamp/now()获取当前时间

2018-04-09 15:20:49.247

二、从日期时间中提取字段

1.year,month,day/dayofmonth,hour,minute,second

Examples:> `SELECT day('2009-07-30');` 30

2.dayofweek (1 = Sunday, 2 = Monday, ..., 7 = Saturday),dayofyear

Examples:> `SELECT dayofweek('2009-07-30');` 5

Since: 2.3.0

3.weekofyear

weekofyear(date) - Returns the week of the year of the given date. A week is considered to start on a Monday and week 1 is the first week with > 3 days.

Examples:> `SELECT weekofyear('2008-02-20');` 8

4.trunc截取某部分的日期，其他部分默认为01

第二个参数 ["year", "yyyy", "yy", "mon", "month", "mm"]

Examples:

```
> SELECT trunc('2009-02-12', 'MM');
2009-02-01
> SELECT trunc('2015-10-27', 'YEAR');
2015-01-01
```

5.date_trunc ("YEAR", "YYYY", "YY", "MON", "MONTH", "MM", "DAY", "DD", "HOUR", "MINUTE", "SECOND", "WEEK", "QUARTER")

Examples:> `SELECT date_trunc('2015-03-05T09:32:05.359', 'HOUR');` 2015-03-05T09:00:00

Since: 2.3.0

6.date_format将时间转化为某种格式的字符串

Examples:> `SELECT date_format('2016-04-08', 'y');` 2016

三、日期时间转换

1.unix_timestamp返回当前时间的unix时间戳

Examples:

```
> SELECT unix_timestamp();1476884637
> SELECT unix_timestamp('2016-04-08', 'yyyy-MM-dd'); 1460041200
```

2.from_unixtime将时间戳换算成当前时间, to_unix_timestamp将时间转化为时间戳

Examples:

```
> SELECT from_unixtime(0, 'yyyy-MM-dd HH:mm:ss');1970-01-01 00:00:00
> SELECT to_unix_timestamp('2016-04-08', 'yyyy-MM-dd');1460041200
```

3.to_date/date将字符串转化为日期格式, to_timestamp (Since: 2.2.0)

```
> SELECT to_date('2009-07-30 04:17:52');2009-07-30
> SELECT to_date('2016-12-31', 'yyyy-MM-dd'); 2016-12-31
> SELECT to_timestamp('2016-12-31 00:12:00'); 2016-12-31 00:12:00
```

4.quarter 将1年4等分(range 1 to 4)

Examples:> `SELECT quarter('2016-08-31');` 3

四、日期、时间计算

1.months_between两个日期之间的月数

months_between(timestamp1, timestamp2) - Returns number of months between timestamp1 and timestamp2.

Examples:> `SELECT months_between('1997-02-28 10:30:00', '1996-10-30');` 3.94959677

2.add_months返回日期后n个月后的日期

Examples:> `SELECT add_months('2016-08-31', 1);` 2016-09-30

3.last_day(date),next_day(start_date, day_of_week)

Examples:

```
> SELECT last_day('2009-01-12');2009-01-31
> SELECT next_day('2015-01-14', 'TU');2015-01-20
```

4.date_add,date_sub(减)

date_add(start_date, num_days) - Returns the date that is num_days after start_date.

Examples:

```
> SELECT date_add('2016-07-30', 1);2016-07-31
```

5.datediff (两个日期期间的天数)

datediff(endDate, startDate) - Returns the number of days from startDate to endDate.

Examples:> SELECT datediff('2009-07-31', '2009-07-30'); 1

6.关于UTC时间

to_utc_timestamp

to_utc_timestamp(timestamp, timezone) - Given a timestamp like '2017-07-14 02:40:00.0', interprets it as a time in the given time zone, and renders that time as a timestamp in UTC. For example, 'GMT+1' would yield '2017-07-14 01:40:00.0'.

Examples:> SELECT to_utc_timestamp('2016-08-31', 'Asia/Seoul');2016-08-30 15:00:00

from_utc_timestamp

from_utc_timestamp(timestamp, timezone) - Given a timestamp like '2017-07-14 02:40:00.0', interprets it as a time in UTC, and renders that time as a timestamp in the given time zone. For example, 'GMT+1' would yield '2017-07-14 03:40:00.0'.

Examples:> SELECT from_utc_timestamp('2016-08-31', 'Asia/Seoul');2016-08-31 09:00:00

7. 计算

2个日期小时差

(unix_timestamp(qingjie_reg_time)-unix_timestamp(real_send_time))/3600

- 返回当前时间下再增加num_months个月的日期

add_months(string start_date, int num_months)

- 返回这个月的最后一天的日期, 忽略时分秒部分 (HH:mm:ss)

last_day(string date)

- 本月1日

写法1: get_date(-day(get_date(-1)))

写法2: concat(substr(get_date(-1), 1, 8), '01')

上月最后一天

get_date(-day(get_date(-1))-1)

- 上月1日

concat(substr(get_date(-day(get_date(-1))-1), 1, 8), '01')

- 返回当前时间的下一个星期X所对应的日期

如: next_day('2015-01-14' , 'TU') = 2015-01-20 以2015-01-14为开始时间, 其下一个星期二所对应的日期为2015-01-20

next_day(string start_date, string day_of_week)

- 返回时间的最开始年份或月份

如trunc("2016-06-26", "MM")=2016-06-01 trunc("2016-06-26", "YY")=2016-01-01

注意所支持的格式为MONTH/MON/MM, YEAR/YYYY/YY trunc(string date, string format)

select concat(substr('2018-08-31' , 1, 7), ' -01')

- 返回date1与date2之间相差的月份,

如date1>date2, 则返回正, 如果date1<date2,则返回负, 否则返回0.0

如: months_between('1997-02-28 10:30:00' , '1996-10-30') = 3.94959677 1997-02-28 10:30:00与1996-10-30相差3.94959677个月

months_between(date1, date2)

- 某日期所在的周一和周日日期

select date_sub('2017-08-06' ,IF(pmod(datediff('2017-08-06' , '1920-01-01')-3, 7))=' 0' , 7, pmod(datediff('2017-08-06' , '1920-01-01')-3, 7))-7)

2. 格式

- 按指定格式返回时间date

如: date_format("2016-06-22"," MM-dd")=06-22

date_format(date/timestamp/string ts, string fmt)

- 将时间的秒值转换成format格式

(format可为 "yyyy-MM-dd hh:mm:ss" , "yyyy-MM-dd hh" , "yyyy-MM-dd hh:mm" 等等)

如from_unixtime(1250111000," yyyy-MM-dd")得到2009-03-12

from_unixtime(bigint unixtime[, string format])

- 将格式为yyyy-MM-dd HH:mm:ss的时间字符串转换成时间戳

如unix_timestamp('2009-03-20 11:30:01') = 1237573801

unix_timestamp(string date)

- 将指定时间字符串格式字符串转换成Unix时间戳, 如果格式不对返回0

如: unix_timestamp('2009-03-20' , 'yyyy-MM-dd') = 1237532400

unix_timestamp(string date, string pattern)

- 返回时间字符串的日期部分

to_date(string timestamp)

- 返回时间字符串的年份部分

year(string date)

- 返回当前时间属性哪个季度

如quarter('2015-04-08') = 2 quarter(date/timestamp/string)

- 返回时间字符串的月份部分

month(string date)
• 返回时间字符串的天

day(string date) dayofmonth(date)
• 返回时间字符串的小时

hour(string date)
• 返回时间字符串的分钟

minute(string date)
• 返回时间字符串的秒

second(string date)
• 返回时间字符串位于一年中的第几个周内

weekofyear(string date)
• 如果给定的时间戳并非UTC，则将其转化成指定的时区下时间戳

from_utc_timestamp(timestamp, string timezone)
• 如果给定的时间戳指定的时区下时间戳，则将其转化成UTC下的时间戳

to_utc_timestamp(timestamp, string timezone)

6、根据两个日期，客户数相减

2021年11月24日 10:04

```
select  
count(distinct if(stat_date='20211110',acct_no,null))-count(distinct if(stat_date='20211031',acct_no,null))  
As million_change  
from fdm_dpa.fin_bof_hold_size_list
```

7、group by 与聚合函数

2021年11月25日 16:03

使用Group By子句的时候，一定要记住下面的一些规则：

- (1) 不能Group By非标量基元类型的列，如不能Group By text, image或bit类型的列
- (2) Select指定的每一列都应该出现在Group By子句中，除非对这一列使用了聚合函数；
- (3) 不能Group By在表中不存在的列；
- (4) 进行分组前可以使用Where子句消除不满足条件的行；
- (5) 使用Group By子句返回的组没有特定的顺序，可以使用Order By子句指定次序。

```
select t1.page_name,t1.count_no (这里修改为sum (t1.count_no) 即可)
from
(select
    page_id
    ,page_name
    ,count(distinct acct_no) as count_no
from fdm_sor.t_fsa_bhvr_new_d
where stat_date='20211122'
and user_bhvr_type='01' --访问
group by page_id,page_name
) t1
inner join
(
select
    page_id
from fdm_sor.t_uba_cfg_page_info_d
where buiz_belong='02'---理财
and buiz_detail in ('0200F','0200E','0200D','0200C','0200B','0200A','02003','0201E')--基金
group by page_id
) t2 on t1.page_id=t2.page_id
group by t1.page_name (select中列必须出现在group by 中，除非是聚合函数)
limit 10
```

8、nvl () 函数--转换null值

2021年11月25日

16:54

NVL (string, replace_with) : 将NULL值转换为某一实际值, 其中string与replace_with的数据类型必须保持一致。

```
Select NVL(type,'没有分类') as '丛书系列名称'
From bookinfo
Group by NVL(type,'没有分类')
```

实例:

```
select nvl(eleid,'无'),sum(hold_amt)
from fdm_dpa.fin_bof_trade_size_list
where stat_date='20211122'
group by nvl(eleid,'无')
```

9、over () 函数--开窗函数

2021年11月25日 17:09

一、开窗函数和聚合函数的含义

1、开窗函数的定义

它和聚合函数是一样的，都是对行的集合组进行聚合计算。它用于为行定义一个窗口（这里的窗口是指运算将要操作的行的集合），它对一组值进行操作，不需要使用GROUP BY子句对数据进行分组，能够在同一行中同时返回基础行的列和聚合列。反正我理解这个函数已经使用好子查询或者是其它方式求得聚合列的值给我合并。

但是与聚合函数不同的是，开窗函数在聚合函数后增加了一个OVER关键字。

2、开窗函数

①、格式

函数名(列) OVER(选项)

②、分类

第一大类：**聚合开窗函数**====>聚合函数(列) OVER (选项)，这里的选项可以是PARTITION BY子句，但不可是ORDER BY子句

第二大类：**排序开窗函数**====>排序函数(列) OVER(选项)，这里的选项可以是ORDER BY子句，也可以是 OVER (PARTITION BY子句 ORDER BY子句)，但不可以是PARTITION BY子句

二、开窗函数的具体介绍---聚合开窗函数和排序开窗函数

1、聚合开窗函数

OVER 关键字表示把聚合函数当成聚合开窗函数而不是聚合函数。**SQL 标准允许将所有聚合函数用做聚合开窗函数。**在上边的例子中，开窗函数COUNT(*) OVER()对于查询结果的每一行都返回所有符合条件的行的条数。OVER关键字后的括号中还经常添加选项用以改变进行聚合运算的窗口范围。如果OVER关键字后的括号中的选项为空，则开窗函数会对结果集中的所有行进行聚合运算。

开窗函数的OVER关键字后括号中的可以使用PARTITION BY 子句来定义行的分区来供进行聚合计算。**与GROUP BY 子句不同，PARTITION BY 子句创建的分区是独立于结果集的，创建的分区只是供进行聚合计算的，**而且不同的开窗函数所创建的分区也不互相影响。下面的SQL语句用于显示每一个人员的信息以及所属城市的人员数：

```
1. SELECT FName, FCITY, FAGE, FSalary, //姓名、城市人员数、年龄、薪水
2. COUNT(FName) OVER(PARTITION BY FCITY)
3. FROM T_Person
```

OVER(PARTITION BY FCITY)表示对结果集按照FCITY进行分区，并且计算当前行所属的组的聚合计算结果。在同一个SELECT语句中可以同时使用多个开窗函数，而且这些开窗函数并不会相互干扰。比如下面的SQL语句用于显示每一个人员的信息、所属城市的人员数以及同龄人的人数：

```
1. SELECT FName, FCITY, FAGE, FSalary,
2. COUNT(FName) OVER(PARTITION BY FCITY),
3. COUNT(FName) OVER(PARTITION BY FAGE)
4. FROM T_Person
```

2、排序开窗函数

对于排序开窗函数来讲，它支持的开窗函数分别为：ROW_NUMBER（行号）、RANK（排名）、DENSE_RANK（密集排名）和NTILE（分组排名）。

```
1. select FName, FSalary, FCity, FAge,
2. row_number() over(order by FSalary) as rownum,
3. rank() over(order by FSalary) as rank,
4. dense_rank() over(order by FSalary) as dense_rank,
5. ntile(6) over(order by FSalary) as ntile
6. from T_Person
7. order by FName
```

	FName	FSalary	FCity	FAge	rownum	rank	dense_rank	ntile
1	Bill	2000	Beijing	25	2	2	2	1
2	Guo	2800	NewYork	20	5	5	3	2
3	Jerry	3300	NewYork	24	10	10	5	4
4	Jim	3500	Beijing	22	12	11	6	5
5	John	1000	NewYork	22	1	1	1	1
6	Ketty	8500	London	25	15	15	9	6
					9	6	4	3
					3	2	2	1
9	Merry	3500	Beijing	23	11	11	6	4
10	Smith	3000	ChengDu	30	7	6	4	3
11	Swing	2000	London	22	4	2	2	2
12	Tim	4000	ChengDu	21	13	13	7	5
13	Tom	3000	Beijing	20	6	6	4	2
14	YaoMing	3000	Beijing	20	8	6	4	3
15	YuQian	8000	Beijing	24	14	14	8	6

按FSalary字段排序得出来的序号

按FSalary字段排序得出来的排名

按FSalary字段排序得出来的密级排名

按FSalary字段排序得出来的分组号

查询已成功执行。

- ①、对于row_number() over(order by FSalary) as rownum来说，这个排序开窗函数是按FSalary升序的方式来排序，并得出排序结果的序号
 - ②、对于rank() over(order by FSalary) as rank来说，这个排序开窗函数是按FSalary升序的方式来排序，并得出排序结果的排名号。这个函数求出来的排名结果可以排列，并列排名之后的排名将是并列的排名加上并列数（简单说每个人只有一种排名，然后出现两个并列第一名的情况，这时候排在两个第一名后面的人将是第三名，也就是没有了第二名，但是有两个第一名）
 - ③、对于dense_rank() over(order by FSalary) as dense_rank来说，这个排序函数是按FSalary升序的方式来排序，并得出排序结果的排名号。这个函数与rank()函数不同在于，并列排名之后的排名只是并列排名加1（简单说每个人只有一种排名，然后出现两个并列第一名的情况，这时候排在两个第一名后面的人将是第二名，也就是两个第一名，一个第二名）
 - ④、对于ntile(6) over(order by FSalary)as ntile 来说，这个排序函数是按FSalary升序的方式来排序，然后6等分成6个组吗，并显示所在组的序号。
- 排序函数和聚合开窗函数类似，也支持在OVER子句中使用PARTITION BY语句。例如：

```

1. select FName, FSalary, FCity, FAge,
2. row_number() over(partition by FName order by FSalary) as rownum,
3. rank() over(partition by FName order by FSalary) as rank,
4. dense_rank() over(partition by FName order by FSalary) as dense_rank,
5. ntile(6) over(partition by FName order by FSalary)as ntile
6. from T_Person
order by FName

```

4、窗口函数

(1) 窗口函数的结构：

分析函数 (max()/sum()/row_number()) + 窗口子句 (over函数)

例：row_number() over(partition by uid order by create_time asc)

(2) 窗口函数应用场景：

Top N

分区排序 —— row_number()

动态group by

累计计算

层次查询

(3) 常用的窗口函数

RANK()：在分组中排名，相同排名时会留下空位；

DENSE_RANK()：在分组中排名，相同排名时不会留下空位；

FIRST_VALUE()：分组内排序取第一个值；

LAST_VALUE()：分组内排序取最后一个值；

NTILE(n)：将分组数据按顺序切分成n份，返回当前所在切片；

ROW_NUMBER()：在分组中从1开始按序记录序列；

CUME_DIST()：小于等于当前值的行数 / 分组总行数（百分比）；

PERCENT_RANK()：（分组内的RANK值-1） / （分组内总数-1）；

LAG(col, n, DEFAULT)：在统计窗口内从下往上取第n行的值；

LEAD(col, n, DEFAULT)：在统计窗口内从上往下取第n行的值；

10、显示hive表的分区

2021年11月29日 11:41

```
show partitions FDM_DPA.FIN_BOF_TRADE_SIZE_LIST
```

显示交易表中的分区

11、判断是不是有缺失的编号

2021年12月1日 14:49

SeqTbl

seq (连续编号)	name (名字)
1	迪克
2	安
3	莱露
5	卡
6	玛丽
8	本

Select

'有缺失的编号' as gap

From SeqTbl

Having count (1) <>max(seq)

如果有缺失的编号，会筛选出结果。

■ 执行结果

gap

·存在缺失的编号·

如果这个查询结果有 1 行，说明存在缺失的编号；如果 1 行都没有，说明不存在缺失的编号。这是因为，如果用 COUNT(*) 统计出来的行数等于“连续编号”列的最大值，就说明编号从开始到最后是连续递增的，中间没有缺失。如果有缺失，COUNT(*) 会小于 MAX(seq)，这样 HAVING 子句就变成真了。这个解法只需要 3 行代码，十分优雅。

二、查询一下缺失编号的最小值

Select min(seq+1)

From SeqTbl

Where (seq+1) not in (select seq from SeqTbl)

12、SQL求众数

2021年12月1日 15:02

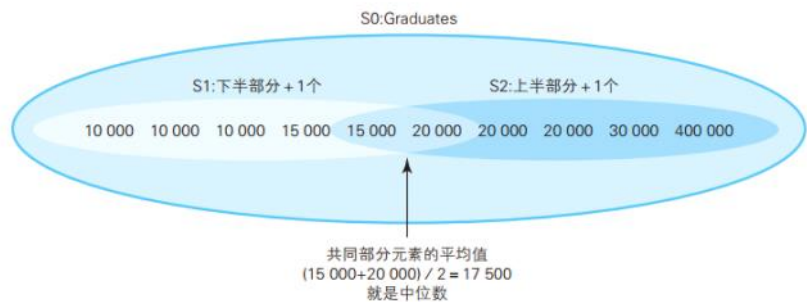
```
select
    prch_amt/100 as prch_amt
    ,COUNT(1) as cnt
from FDM_DM.DM_FIN_DAILY_RPT_D
group by prch_amt
having cnt >= (select max(cnt) from (select prch_amt/100 as prch_amt,COUNT(1) as
cnt from FDM_DM.DM_FIN_DAILY_RPT_D group by prch_amt))---最大值
```

13、SQL求中位数

2021年12月1日 15:52

做法是，将集合里的元素按照大小分为上半部分和下半部分两个子集，同时让这 2 个子集共同拥有集合正中间的元素。这样，共同部分的元素的平均值就是中位数，思路如下图所示。

■ 中位数求法的思路



--求中位数的SQL语句：在HAVING子句中使用非等值自连接

```
SELECT AVG(DISTINCT income)
```

```
FROM (SELECT T1.income
```

```
FROM Graduates T1, Graduates T2
```

```
GROUP BY T1.income
```

--S1的条件

```
HAVING SUM(CASE WHEN T2.income >= T1.income THEN 1 ELSE 0 END)
```

```
>= COUNT(*) / 2
```

--S2的条件

```
AND SUM(CASE WHEN T2.income <= T1.income THEN 1 ELSE 0 END)
```

```
>= COUNT(*) / 2 ) TMP;
```

14、substring_index含义

2021年12月10日 14:49

定义

SUBSTRING_INDEX - 按分隔符截取字符串

语法

SUBSTRING_INDEX(str, delimiter, count)

返回一个 str 的子字符串，在 delimiter 出现 count 次的位置截取。

如果 count > 0，从则左边数起，且返回位置前的子串；

如果 count < 0，从则右边数起，且返回位置后的子串。

delimiter 是大小写敏感，且是多字节安全的。

示例

```
SELECT SUBSTRING_INDEX('www.mysql.com', '.', 2);
```

```
-> 'www.mysql'
```

```
SELECT SUBSTRING_INDEX('www.mysql.com', '.', -2);
```

```
-> 'mysql.com'
```

15、coalesce () 函数

2021年12月10日 15:23

coalesce：聚合，联合，结合。

①用途：

将空值替换成其他值

返回第一个非空值

②表达式：

COALESCE是一个函数，(expression_1, expression_2, ...,expression_n)依次参考各参数表达式，遇到非null值即停止并返回该值。如果所有的表达式都是空值，最终将返回一个空值。使用COALESCE在于大部分包含空值的表达式最终将返回空值。

③SQL实例：

```
select coalesce(success_cnt, 1) from tableA
```

当success_cnt 为null值的时候，将返回1，否则将返回success_cnt的真实值。

```
select coalesce(success_cnt,period,1) from tableA
```

当success_cnt不为null，那么无论period是否为null，都将返回success_cnt的真实值（因为success_cnt是第一个参数），当success_cnt为null，而period不为null的时候，返回period的真实值。只有当success_cnt和period均为null的时候，将返回1。

16、判断数据唯一性字段

2021年12月20日 16:41

```
select count(distinct order_no),count( order_no)
from fdm_dpa.fin_bof_trade_size_list
where stat_date='20211219'
and pd_type_cd='01'
and order_type='申购'
```



超时时间: 5分钟  是否解密: 否 

```
1 select count(distinct order_no),count( order_no)
2 from fdm_dpa.fin_bof_trade_size_list
3 where stat_date='20211219'
4 and pd_type_cd='01'
5 and order_type='申购'
```

日志		结果	查看导出手册 最大行数: 1000 		
count(DISTINCT order_no)		count(order_no)			
105430		105430			

17、求月日均数据

2021年12月21日 11:21

```
SELECT
    '市场营销部'          AS BUSINESS_LINE_NAME
    , '004'                AS KPI_CODE
    , '理财日均申购客户数' AS KPI_NAME
    , floor(sum(t.c1)/substr('${statisdate}',7,2)) AS KPI_VALUE
FROM
(
    SELECT
        stat_date, count(distinct acct_no) as c1
    FROM   FDM_DPA.FIN_BOF_TRADE_SIZE_LIST
    WHERE  substr(stat_date,1,6)=substr('${statisdate}',1,6)
    AND    stat_date <= '${statisdate}'
    AND    pd_type_cd='01'
    AND    PD_TYPE_LEVEL_2 <> '定期产品'
    AND    order_type='申购'
    group by stat_date
) t
```

18、instr()函数

2022年1月28日 17:35

instr函数为字符查找函数，其功能是查找一个字符串在另一个字符串中首次出现的位置。

语法

instr(string1, string2, start_position,nth_appearance)

参数

- string1：源字符串，要在此字符串中查找。
- string2：要在string1中查找的字符串。
- start_position：代表string1 的哪个位置开始查找。此参数可选，如果省略默认为1. 字符串索引从1开始。如果此参数为正，从左到右开始检索，如果此参数为负，从右到左检索，返回要查找的字符串在源字符串中的开始索引。
- nth_appearance：代表要查找第几次出现的string2. 此参数可选，如果省略，默认为 1. 如果为负数系统会报错

示例

```
1 select instr('helloworld','l') from dual; --返回结果: 3      默认第一次出现 "l"的位置
2 select instr('helloworld','lo') from dual; --返回结果: 4      即: 在 "lo"中, "l"开始出现的位置
3 select instr('helloworld','wo') from dual; --返回结果:
select * from fdm_dpa.fin_sa_marketing_analyse_d
where stat_date='20220322'
--and (is_activate='是')
and acct_no='00000000000016944582'
6      即 "w"开始出现的位置
```

19、get_json_object和str_to_map解析

2022年3月14日 15:58

1、get_json_object 用于处理json格式数据的函数

使用get_json_object函数从json格式数据中解析出name字段信息：

```
select acct_no
      ,get_json_object(elmn_ctnt_id,'$.eleid') as eleid
      ,get_json_object(elmn_ctnt_id,'$.lccode') as lccode
from FDM_SOR.T_FSA_BHVR_NEW_D
where USER_BHVR_TYPE in ('02','03')
and stat_date='20220124'
group by acct_no
```

2、STR_TO_MAP函数用于处理文本型数据

(1) 语法描述

STR_TO_MAP(VARCHAR text, VARCHAR listDelimiter, VARCHAR keyValueDelimiter)

(2) 功能描述

使用listDelimiter将text分隔成K-V对，然后使用keyValueDelimiter分隔每个K-V对，组装成MAP返回。默认listDelimiter为（，），keyValueDelimiter为（=）。

(3) 获取KEY对应的value值

取用map里的字段，用[""]即可

(4) 案例

4.1

```
str_to_map('1001=2020-06-14,1002=2020-06-14', ',' , '=')
```

输出 {"1001":"2020-06-14","1002":"2020-06-14"}

4.2

```
select str_to_map(regexp_replace(NVL(click_pstn_ctnt,''),' ',''),'\v',':')["lccode"] as lccode,*
from FDM_SOR.T_FSA_BHVR_NEW_D
where stat_date='20220307'
```


20、SQL执行顺序

2022年1月28日 17:35

一、sql执行顺序

- (1) from
- (3) join
- (2) on
- (4) where
- (5) group by (开始使用select中的别名，后面的语句中都可以使用)
- (6) avg, sum, ...
- (7) having
- (8) select
- (9) distinct
- (10) order by

从这个顺序中我们不难发现，所有的查询语句都是从from开始执行的，在执行过程中，每个步骤都会为下一个步骤生成一个虚拟表，这个虚拟表将作为下一个执行步骤的输入。

第一步：首先对from子句中的前两个表执行一个笛卡尔乘积，此时生成虚拟表 vt1（选择相对小的表做基础表）

第二步：接下来便是应用on筛选器，on 中的逻辑表达式将应用到 vt1 中的各个行，筛选出满足on逻辑表达式的行，生成虚拟表 vt2

第三步：如果是outer join 那么这一步就将添加外部行，left outer join 就把左表在第二步中过滤的添加进来，如果是right outer join 那么就将右表在第二步中过滤掉的行添加进来，这样生成虚拟表 vt3

第四步：如果 from 子句中的表数目多余两个表，那么就将vt3和第三个表连接从而计算笛卡尔乘积，生成虚拟表，该过程就是一个重复1-3的步骤，最终得到一个新的虚拟表 vt3。

第五步：应用where筛选器，对上一步生产的虚拟表引用where筛选器，生成虚拟表vt4，在这有个比较重要的细节不得不说一下，对于包含outer join子句的查询，就有一个让人感到困惑的问题，到底在on筛选器还是用where筛选器指定逻辑表达式呢？on和where的最大区别在于，如果在on应用逻辑表达式那么在第三步outer join中还可以把移除的行再次添加回来，而where的移除的最终。举个简单的例子，有一个学生表（班级,姓名）和一个成绩表(姓名,成绩)，我现在需要返回一个x班级的全体同学的成绩，但是这个班级有几个学生缺考，也就是说在成绩表中没有记录。为了得到我们预期的结果我们就需要在on子句指定学生和成绩表的关系（学生.姓名=成绩.姓名）那么我们是否发现在执行第二步的时候，对于没有参加考试的学生记录就不会出现在vt2中，因为他们被on的逻辑表达式过滤掉了,但是我们用left outer join就可以把左表（学生）中没有参加考试的学生找回来，因为我们想返回的是x班级的所有学生，如果在on中应用学生.班级='x'的话，left outer join会把x班级的所有学生记录找回（感谢网友康钦谋_康钦苗的指正），所以只能在where筛选器中应用学生.班级='x' 因为它的过滤是最终的。

第六步：group by 子句将中的唯一的值组合成为一组，得到虚拟表vt5。如果应用了group by，那么后面的所有步骤都只能得到的vt5的列或者是聚合函数（count、sum、avg等）。原因在于最终的结果集中只为每个组包含一行。这一点请牢记。

第七步：应用cube或者rollup选项，为vt5生成超组，生成vt6。

第八步：应用having筛选器，生成vt7。having筛选器是第一个也是为唯一——一个应用到已分组数据的筛选器。

第九步：处理select子句。将vt7中的在select中出现的列筛选出来。生成vt8。

第十步：应用distinct子句，vt8中移除相同的行，生成vt9。事实上如果应用了group by子句那么distinct是多余的，原因同样在于，分组的时候是将列中唯一的值分成一组，同时只为每一组返回一行记录，那么所以的记录都将是不相同的。

第十一步：应用order by子句。按照order_by_condition排序vt9，此时返回的一个游标，而不是虚拟表。sql是基于集合的理论的，集合不会预先对他的行排序，它只是成员的逻辑集合，成员的顺序是无关紧要的。对表进行排序的查询可以返回一个对象，这个对象包含特定的物理顺序的逻辑组织。这个对象就叫游标。正因为返回值是游标，那么使用order by 子句查询不能应用于表表达式。排序是很需要成本的，除非你必须要排序，否则最好不要指定order by，最后，在这一步中是第一个也是唯一——一个可以使用select列表中别名的步骤。

第十二步：应用top选项。此时才返回结果给请求者即用户。

21、limit和offset用法

2022年4月2日 10:20

limit和offset用法

mysql里分页一般用limit来实现

1. select* from article LIMIT 1,3

2.select * from article LIMIT 3 OFFSET 1

上面两种写法都表示取2,3,4三条数据

当limit后面跟两个参数的时候，第一个数表示要跳过的数量，后一位表示要取的数量,例如

select* from article LIMIT 1,3 就是跳过1条数据,从第2条数据开始取，取3条数据，也就是取2,3,4三条数据

当 limit后面跟一个参数的时候，该参数表示要取的数据的数量

例如 select* from article LIMIT 3 表示直接取前三条数据，类似sqlserver里的top语法。

当 limit和offset组合使用的时候，limit后面只能有一个参数，表示要取的的数量,offset表示要跳过的数量。

例如select * from article LIMIT 3 OFFSET 1 表示跳过1条数据,从第2条数据开始取，取3条数据，也就是取2,3,4三条数据

22、保留小数操作

2022年8月22日 15:21

- 1、`cast(sum(hold_amt)/100 as decimal(20,2))`
- 2、`round (hold_amt,2)`

1、批量将中文改成拼音

2021年12月7日 19:14

步骤1: 打开Excel->工具->宏->Viuual Basic编辑器
在弹出来的窗口中对着VBaproject点右键->插入->模块
下面会出现一个名为"模块1", 点击
在右边的空白栏中粘贴以下内容:

```
Function pinyin(p As String) As String
i = Asc(p)
Select Case i
Case -20319 To -20318: pinyin = "a"
Case -20317 To -20305: pinyin = "ai"
Case -20304 To -20296: pinyin = "an"
Case -20295 To -20293: pinyin = "ang"
Case -20292 To -20284: pinyin = "ao"
Case -20283 To -20266: pinyin = "ba"
Case -20265 To -20258: pinyin = "bai"
Case -20257 To -20243: pinyin = "ban"
Case -20242 To -20231: pinyin = "bang"
Case -20230 To -20052: pinyin = "bao"
Case -20051 To -20037: pinyin = "bei"
Case -20036 To -20033: pinyin = "ben"
Case -20032 To -20027: pinyin = "beng"
Case -20026 To -20003: pinyin = "bi"
Case -20002 To -19991: pinyin = "bian"
Case -19990 To -19987: pinyin = "biao"
Case -19986 To -19983: pinyin = "bie"
Case -19982 To -19977: pinyin = "bin"
Case -19976 To -19806: pinyin = "bing"
Case -19805 To -19785: pinyin = "bo"
Case -19784 To -19776: pinyin = "bu"
Case -19775 To -19775: pinyin = "ca"
Case -19774 To -19764: pinyin = "cai"
Case -19763 To -19757: pinyin = "can"
Case -19756 To -19752: pinyin = "cang"
Case -19751 To -19747: pinyin = "cao"
Case -19746 To -19742: pinyin = "ce"
Case -19741 To -19740: pinyin = "ceng"
Case -19739 To -19729: pinyin = "cha"
Case -19728 To -19726: pinyin = "chai"
Case -19725 To -19716: pinyin = "chan"
Case -19715 To -19541: pinyin = "chang"
Case -19540 To -19532: pinyin = "chao"
Case -19531 To -19526: pinyin = "che"
Case -19525 To -19516: pinyin = "chen"
Case -19515 To -19501: pinyin = "cheng"
Case -19500 To -19485: pinyin = "chi"
Case -19484 To -19480: pinyin = "chong"
Case -19479 To -19468: pinyin = "chou"
Case -19467 To -19290: pinyin = "chu"
Case -19289 To -19289: pinyin = "chuai"
Case -19288 To -19282: pinyin = "chuan"
Case -19281 To -19276: pinyin = "chuang"
Case -19275 To -19271: pinyin = "chui"
```

步骤2: 在需要完成的单元格中输入函数：=getpy(文字所在单元格), 其余向下拉即可。

用于复制代码:

https://blog.csdn.net/qq_39783601/article/details/109851172?ops_request_misc=%257B%2522request%255Fid%2522%253A%2522163887557516780264014829%2522%252C%2522scm%2522%253A%25220140713.130102334..%2522%257D&request_id=163887557516780264014829&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduend~default-1-109851172.first_rank_v2~pc_rank_v29&utm_term=excel+%E4%B8%AD%E6%96%87%E8%BD%AC%E6%8D%A2%E6%88%90%E6%8B%BC%E9%9F%B3&spm=1018.2226.3001.4187

Case -19270 To -19264: pinyin = "chun"
Case -19263 To -19262: pinyin = "chuo"
Case -19261 To -19250: pinyin = "ci"
Case -19249 To -19244: pinyin = "cong"
Case -19243 To -19243: pinyin = "cou"
Case -19242 To -19239: pinyin = "cu"
Case -19238 To -19236: pinyin = "cuan"
Case -19235 To -19228: pinyin = "cui"
Case -19227 To -19225: pinyin = "cun"
Case -19224 To -19219: pinyin = "cuo"
Case -19218 To -19213: pinyin = "da"
Case -19212 To -19039: pinyin = "dai"
Case -19038 To -19024: pinyin = "dan"
Case -19023 To -19019: pinyin = "dang"
Case -19018 To -19007: pinyin = "dao"
Case -19006 To -19004: pinyin = "de"
Case -19003 To -18997: pinyin = "deng"
Case -18996 To -18978: pinyin = "di"
Case -18977 To -18962: pinyin = "dian"
Case -18961 To -18953: pinyin = "diao"
Case -18952 To -18784: pinyin = "die"
Case -18783 To -18775: pinyin = "ding"
Case -18774 To -18774: pinyin = "diu"
Case -18773 To -18527: pinyin = "dong"
Case -18526 To -18519: pinyin = "fa"
Case -18518 To -18502: pinyin = "fan"
Case -18501 To -18491: pinyin = "fang"
Case -18490 To -18479: pinyin = "fei"
Case -18478 To -18464: pinyin = "fen"
Case -18463 To -18449: pinyin = "feng"
Case -18448 To -18448: pinyin = "fo"
Case -18447 To -18447: pinyin = "fou"
Case -18446 To -18240: pinyin = "fu"
Case -18239 To -18238: pinyin = "ga"
Case -18237 To -18232: pinyin = "gai"
Case -18231 To -18221: pinyin = "gan"
Case -18220 To -18212: pinyin = "gang"
Case -18211 To -18202: pinyin = "gao"
Case -18201 To -18185: pinyin = "ge"
Case -18184 To -18184: pinyin = "gei"
Case -18183 To -18182: pinyin = "gen"
Case -18181 To -18013: pinyin = "geng"
Case -18012 To -17998: pinyin = "gong"
Case -17997 To -17989: pinyin = "gou"
Case -17988 To -17971: pinyin = "gu"
Case -17970 To -17965: pinyin = "gua"
Case -17964 To -17962: pinyin = "guai"
Case -17961 To -17951: pinyin = "guan"
Case -17950 To -17948: pinyin = "guang"
Case -17947 To -17932: pinyin = "gui"
Case -17931 To -17929: pinyin = "gun"
Case -17928 To -17923: pinyin = "guo"
Case -17922 To -17760: pinyin = "ha"
Case -17759 To -17753: pinyin = "hai"
Case -17752 To -17734: pinyin = "han"
Case -17733 To -17731: pinyin = "hang"
Case -17730 To -17722: pinyin = "hao"

Case -17721 To -17704: pinyin = "he"
Case -17703 To -17702: pinyin = "hei"
Case -17701 To -17698: pinyin = "hen"
Case -17697 To -17693: pinyin = "heng"
Case -17692 To -17684: pinyin = "hong"
Case -17683 To -17677: pinyin = "hou"
Case -17676 To -17497: pinyin = "hu"
Case -17496 To -17488: pinyin = "hua"
Case -17487 To -17483: pinyin = "huai"
Case -17482 To -17469: pinyin = "huan"
Case -17468 To -17455: pinyin = "huang"
Case -17454 To -17434: pinyin = "hui"
Case -17433 To -17428: pinyin = "hun"
Case -17427 To -17418: pinyin = "huo"
Case -17417 To -17203: pinyin = "ji"
Case -17202 To -17186: pinyin = "jia"
Case -17185 To -16984: pinyin = "jian"
Case -16983 To -16971: pinyin = "jiang"
Case -16970 To -16943: pinyin = "jiao"
Case -16942 To -16916: pinyin = "jie"
Case -16915 To -16734: pinyin = "jin"
Case -16733 To -16709: pinyin = "jing"
Case -16708 To -16707: pinyin = "jiong"
Case -16706 To -16690: pinyin = "jiu"
Case -16689 To -16665: pinyin = "ju"
Case -16664 To -16658: pinyin = "juan"
Case -16657 To -16648: pinyin = "jue"
Case -16647 To -16475: pinyin = "jun"
Case -16474 To -16471: pinyin = "ka"
Case -16470 To -16466: pinyin = "kai"
Case -16465 To -16460: pinyin = "kan"
Case -16459 To -16453: pinyin = "kang"
Case -16452 To -16449: pinyin = "kao"
Case -16448 To -16434: pinyin = "ke"
Case -16433 To -16430: pinyin = "ken"
Case -16429 To -16428: pinyin = "keng"
Case -16427 To -16424: pinyin = "kong"
Case -16423 To -16420: pinyin = "kou"
Case -16419 To -16413: pinyin = "ku"
Case -16412 To -16408: pinyin = "kua"
Case -16407 To -16404: pinyin = "kuai"
Case -16403 To -16402: pinyin = "kuan"
Case -16401 To -16394: pinyin = "kuang"
Case -16393 To -16221: pinyin = "kui"
Case -16220 To -16217: pinyin = "kun"
Case -16216 To -16213: pinyin = "kuo"
Case -16212 To -16206: pinyin = "la"
Case -16205 To -16203: pinyin = "lai"
Case -16202 To -16188: pinyin = "lan"
Case -16187 To -16181: pinyin = "lang"
Case -16180 To -16172: pinyin = "lao"
Case -16171 To -16170: pinyin = "le"
Case -16169 To -16159: pinyin = "lei"
Case -16158 To -16156: pinyin = "leng"
Case -16155 To -15960: pinyin = "li"
Case -15959 To -15959: pinyin = "lia"
Case -15958 To -15945: pinyin = "lian"

Case -15944 To -15934: pinyin = "liang"
Case -15933 To -15921: pinyin = "liao"
Case -15920 To -15916: pinyin = "lie"
Case -15915 To -15904: pinyin = "lin"
Case -15903 To -15890: pinyin = "ling"
Case -15889 To -15879: pinyin = "liu"
Case -15878 To -15708: pinyin = "long"
Case -15707 To -15702: pinyin = "lou"
Case -15701 To -15682: pinyin = "lu"
Case -15681 To -15668: pinyin = "lv"
Case -15667 To -15662: pinyin = "luan"
Case -15661 To -15660: pinyin = "lue"
Case -15659 To -15653: pinyin = "lun"
Case -15652 To -15641: pinyin = "luo"
Case -15640 To -15632: pinyin = "ma"
Case -15631 To -15626: pinyin = "mai"
Case -15625 To -15455: pinyin = "man"
Case -15454 To -15449: pinyin = "mang"
Case -15448 To -15437: pinyin = "mao"
Case -15436 To -15436: pinyin = "me"
Case -15435 To -15420: pinyin = "mei"
Case -15419 To -15417: pinyin = "men"
Case -15416 To -15409: pinyin = "meng"
Case -15408 To -15395: pinyin = "mi"
Case -15394 To -15386: pinyin = "mian"
Case -15385 To -15378: pinyin = "miao"
Case -15377 To -15376: pinyin = "mie"
Case -15375 To -15370: pinyin = "min"
Case -15369 To -15364: pinyin = "ming"
Case -15363 To -15363: pinyin = "miu"
Case -15362 To -15184: pinyin = "mo"
Case -15183 To -15181: pinyin = "mou"
Case -15180 To -15166: pinyin = "mu"
Case -15165 To -15159: pinyin = "na"
Case -15158 To -15154: pinyin = "nai"
Case -15153 To -15151: pinyin = "nan"
Case -15150 To -15150: pinyin = "nang"
Case -15149 To -15145: pinyin = "nao"
Case -15144 To -15144: pinyin = "ne"
Case -15143 To -15142: pinyin = "nei"
Case -15141 To -15141: pinyin = "nen"
Case -15140 To -15140: pinyin = "neng"
Case -15139 To -15129: pinyin = "ni"
Case -15128 To -15122: pinyin = "nian"
Case -15121 To -15120: pinyin = "niang"
Case -15119 To -15118: pinyin = "niao"
Case -15117 To -15111: pinyin = "nie"
Case -15110 To -15110: pinyin = "nin"
Case -15109 To -14942: pinyin = "ning"
Case -14941 To -14938: pinyin = "niu"
Case -14937 To -14934: pinyin = "nong"
Case -14933 To -14931: pinyin = "nu"
Case -14930 To -14930: pinyin = "nv"
Case -14929 To -14929: pinyin = "nuan"
Case -14928 To -14927: pinyin = "nue"
Case -14926 To -14923: pinyin = "nuo"
Case -14922 To -14922: pinyin = "o"

Case -14921 To -14915: pinyin = "ou"
Case -14914 To -14909: pinyin = "pa"
Case -14908 To -14903: pinyin = "pai"
Case -14902 To -14895: pinyin = "pan"
Case -14894 To -14890: pinyin = "pang"
Case -14889 To -14883: pinyin = "pao"
Case -14882 To -14874: pinyin = "pei"
Case -14873 To -14872: pinyin = "pen"
Case -14871 To -14858: pinyin = "peng"
Case -14857 To -14679: pinyin = "pi"
Case -14678 To -14675: pinyin = "pian"
Case -14674 To -14671: pinyin = "piao"
Case -14670 To -14669: pinyin = "pie"
Case -14668 To -14664: pinyin = "pin"
Case -14663 To -14655: pinyin = "ping"
Case -14654 To -14646: pinyin = "po"
Case -14645 To -14631: pinyin = "pu"
Case -14630 To -14595: pinyin = "qi"
Case -14594 To -14430: pinyin = "qia"
Case -14429 To -14408: pinyin = "qian"
Case -14407 To -14400: pinyin = "qiang"
Case -14399 To -14385: pinyin = "qiao"
Case -14384 To -14380: pinyin = "qie"
Case -14379 To -14369: pinyin = "qin"
Case -14368 To -14356: pinyin = "qing"
Case -14355 To -14354: pinyin = "qiong"
Case -14353 To -14346: pinyin = "qiu"
Case -14345 To -14171: pinyin = "qu"
Case -14170 To -14160: pinyin = "quan"
Case -14159 To -14152: pinyin = "que"
Case -14151 To -14150: pinyin = "qun"
Case -14149 To -14146: pinyin = "ran"
Case -14145 To -14141: pinyin = "rang"
Case -14140 To -14138: pinyin = "rao"
Case -14137 To -14136: pinyin = "re"
Case -14135 To -14126: pinyin = "ren"
Case -14125 To -14124: pinyin = "reng"
Case -14123 To -14123: pinyin = "ri"
Case -14122 To -14113: pinyin = "rong"
Case -14112 To -14110: pinyin = "rou"
Case -14109 To -14100: pinyin = "ru"
Case -14099 To -14098: pinyin = "ruan"
Case -14097 To -14095: pinyin = "rui"
Case -14094 To -14093: pinyin = "run"
Case -14092 To -14091: pinyin = "ruo"
Case -14090 To -14088: pinyin = "sa"
Case -14087 To -14084: pinyin = "sai"
Case -14083 To -13918: pinyin = "san"
Case -13917 To -13915: pinyin = "sang"
Case -13914 To -13911: pinyin = "sao"
Case -13910 To -13908: pinyin = "se"
Case -13907 To -13907: pinyin = "sen"
Case -13906 To -13906: pinyin = "seng"
Case -13905 To -13897: pinyin = "sha"
Case -13896 To -13895: pinyin = "shai"
Case -13894 To -13879: pinyin = "shan"
Case -13878 To -13871: pinyin = "shang"

Case -13870 To -13860: pinyin = "shao"
Case -13859 To -13848: pinyin = "she"
Case -13847 To -13832: pinyin = "shen"
Case -13831 To -13659: pinyin = "sheng"
Case -13658 To -13612: pinyin = "shi"
Case -13611 To -13602: pinyin = "shou"
Case -13601 To -13407: pinyin = "shu"
Case -13406 To -13405: pinyin = "shua"
Case -13404 To -13401: pinyin = "shuai"
Case -13400 To -13399: pinyin = "shuan"
Case -13398 To -13396: pinyin = "shuang"
Case -13395 To -13392: pinyin = "shui"
Case -13391 To -13388: pinyin = "shun"
Case -13387 To -13384: pinyin = "shuo"
Case -13383 To -13368: pinyin = "si"
Case -13367 To -13360: pinyin = "song"
Case -13359 To -13357: pinyin = "sou"
Case -13356 To -13344: pinyin = "su"
Case -13343 To -13341: pinyin = "suan"
Case -13340 To -13330: pinyin = "sui"
Case -13329 To -13327: pinyin = "sun"
Case -13326 To -13319: pinyin = "suo"
Case -13318 To -13148: pinyin = "ta"
Case -13147 To -13139: pinyin = "tai"
Case -13138 To -13121: pinyin = "tan"
Case -13120 To -13108: pinyin = "tang"
Case -13107 To -13097: pinyin = "tao"
Case -13096 To -13096: pinyin = "te"
Case -13095 To -13092: pinyin = "teng"
Case -13091 To -13077: pinyin = "ti"
Case -13076 To -13069: pinyin = "tian"
Case -13068 To -13064: pinyin = "tiao"
Case -13063 To -13061: pinyin = "tie"
Case -13060 To -12889: pinyin = "ting"
Case -12888 To -12876: pinyin = "tong"
Case -12875 To -12872: pinyin = "tou"
Case -12871 To -12861: pinyin = "tu"
Case -12860 To -12859: pinyin = "tuan"
Case -12858 To -12853: pinyin = "tui"
Case -12852 To -12850: pinyin = "tun"
Case -12849 To -12839: pinyin = "tuo"
Case -12838 To -12832: pinyin = "wa"
Case -12831 To -12830: pinyin = "wai"
Case -12829 To -12813: pinyin = "wan"
Case -12812 To -12803: pinyin = "wang"
Case -12802 To -12608: pinyin = "wei"
Case -12607 To -12598: pinyin = "wen"
Case -12597 To -12595: pinyin = "weng"
Case -12594 To -12586: pinyin = "wo"
Case -12585 To -12557: pinyin = "wu"
Case -12556 To -12360: pinyin = "xi"
Case -12359 To -12347: pinyin = "xia"
Case -12346 To -12321: pinyin = "xian"
Case -12320 To -12301: pinyin = "xiang"
Case -12300 To -12121: pinyin = "xiao"
Case -12120 To -12100: pinyin = "xie"
Case -12099 To -12090: pinyin = "xin"

Case -12089 To -12075: pinyin = "xing"
Case -12074 To -12068: pinyin = "xiong"
Case -12067 To -12059: pinyin = "xiu"
Case -12058 To -12040: pinyin = "xu"
Case -12039 To -11868: pinyin = "xuan"
Case -11867 To -11862: pinyin = "xue"
Case -11861 To -11848: pinyin = "xun"
Case -11847 To -11832: pinyin = "ya"
Case -11831 To -11799: pinyin = "yan"
Case -11798 To -11782: pinyin = "yang"
Case -11781 To -11605: pinyin = "yao"
Case -11604 To -11590: pinyin = "ye"
Case -11589 To -11537: pinyin = "yi"
Case -11536 To -11359: pinyin = "yin"
Case -11358 To -11341: pinyin = "ying"
Case -11340 To -11340: pinyin = "yo"
Case -11339 To -11325: pinyin = "yong"
Case -11324 To -11304: pinyin = "you"
Case -11303 To -11098: pinyin = "yu"
Case -11097 To -11078: pinyin = "yuan"
Case -11077 To -11068: pinyin = "yue"
Case -11067 To -11056: pinyin = "yun"
Case -11055 To -11053: pinyin = "za"
Case -11052 To -11046: pinyin = "zai"
Case -11045 To -11042: pinyin = "zan"
Case -11041 To -11039: pinyin = "zang"
Case -11038 To -11025: pinyin = "zao"
Case -11024 To -11021: pinyin = "ze"
Case -11020 To -11020: pinyin = "zei"
Case -11019 To -11019: pinyin = "zen"
Case -11018 To -11015: pinyin = "zeng"
Case -11014 To -10839: pinyin = "zha"
Case -10838 To -10833: pinyin = "zhai"
Case -10832 To -10816: pinyin = "zhan"
Case -10815 To -10801: pinyin = "zhang"
Case -10800 To -10791: pinyin = "zhao"
Case -10790 To -10781: pinyin = "zhe"
Case -10780 To -10765: pinyin = "zhen"
Case -10764 To -10588: pinyin = "zheng"
Case -10587 To -10545: pinyin = "zhi"
Case -10544 To -10534: pinyin = "zhong"
Case -10533 To -10520: pinyin = "zhou"
Case -10519 To -10332: pinyin = "zhu"
Case -10331 To -10330: pinyin = "zhua"
Case -10329 To -10329: pinyin = "zhuai"
Case -10328 To -10323: pinyin = "zhuan"
Case -10322 To -10316: pinyin = "zhuang"
Case -10315 To -10310: pinyin = "zhui"
Case -10309 To -10308: pinyin = "zhun"
Case -10307 To -10297: pinyin = "zhuo"
Case -10296 To -10282: pinyin = "zi"
Case -10281 To -10275: pinyin = "zong"
Case -10274 To -10271: pinyin = "zou"
Case -10270 To -10263: pinyin = "zu"
Case -10262 To -10261: pinyin = "zuan"
Case -10260 To -10257: pinyin = "zui"
Case -10256 To -10255: pinyin = "zun"

```
Case -10254 To -10254: pinyin = "zuo"  
Case Else: pinyin = p  
End Select  
End Function  
Function getpy(str)  
For i = 1 To Len(str)  
getpy = getpy & pinyin(Mid(str, i, 1))  
Next i  
End Function
```

2、重要的27个Excel函数公式

2021年12月13日 11:09

重要的27个Excel函数公式



[回忆未来](#)

<https://zhuanlan.zhihu.com/p/94102393>

目录

一、数字处理

1、取绝对值

2、取整

3、四舍五入

二、判断公式

1、把公式产生的错误值显示为空

2、IF多条件判断返回值

三、统计公式

1、统计两个表格重复的内容

2、统计不重复的总人数

四、求和公式

1、隔列求和

2、单条件求和

3、单条件模糊求和

4、多条件模糊求和

5、多表相同位置求和

6、按日期和产品求和

五、查找与引用公式

1、单条件查找公式

2、双向查找公式

3、查找最后一条符合条件的记录。

4、多条件查找

5、指定区域最后一个非空值查找

6、按数字区域间取对应的值

六、字符串处理公式

1、多单元格字符串合并

2、截取除后3位之外的部分

3、截取-前的部分

4、截取字符串中任一段的公式

5、字符串查找

6、字符串查找一对多

七、日期计算公式

1、两日期相隔的年、月、天数计算

2、扣除周末天数的工作日天数

一、数字处理

1、取绝对值

=ABS(数字)

2、取整

=INT(数字)

3、四舍五入

=ROUND(数字, [小数位数](#))

二、判断公式

1、把公式产生的错误值显示为空

公式: C2

=IFERROR(A2/B2,"")

说明: 如果是错误值则显示为空, 否则正常显示。

	C2			
	A	B	C	D
1	实际	计划	完成率	
2	23	67	34%	
3	5	12	42%	
4	89			
5	9	5	180%	
6				

2、IF多条件判断返回值

公式: C2

=IF(AND(A2<500,B2="未到期"),"补款","")

说明: 两个条件同时成立用AND, 任何一个成立用[OR函数](#)。

	C2			
	A	B	C	D
1	金额	是否到期	提醒	
2	800	未到期		
3	200	已到期		
4	300	未到期	补款	
5				
6				

三、统计公式

1、统计两个表格重复的内容

公式: B2

=COUNTIF(Sheet15!A:A,A2)

说明: 如果[返回值](#)大于0说明在另一个表中存在, 0则不存在。

	A	B	C
1	姓名	查找重复	
2	A	0	
3	B	1	
4	C	1	
5	D	1	
6	G	0	
7	Y	0	

	A	B	C
1	姓名		
2	D		
3	C		
4	R		
5	Q		
6	W		
7	B		

2、统计不重复的总人数

公式: C2

=SUMPRODUCT(1/COUNTIF(A2:A8,A2:A8))

说明: 用COUNTIF统计出每人的出现次数, 用1除的方式把出现次数变成分母, 然后相加。

	A	B	C	D
1	姓名		总人数	
2	A		4	
3	B			
4	C			
5	D			
6	A			
7	A			
8	B			

四、求和公式

1、隔列求和

公式：H3

=SUMIF(\$A\$2:\$G\$2,H\$2,A3:G3)

或

=SUMPRODUCT((MOD(COLUMN(B3:G3),2)=0)*B3:G3)

说明：如果标题行没有规则用第2个公式

	A	B	C	D	E	F	G	H	I
1	产品	1月	2月	3月	合计				
2		实际	计划	实际	计划	实际	计划	实际	计划
3	A	1	6	9	3	7	4	17	
4	B	9	7	6	5	2	3	7	
5	C	6	10	10	7	6	5	2	
6									
7									
8									
9									
10									

公式：

=SUMIF(\$A\$2:\$G\$2,H\$2,A3:G3)

2、单条件求和

公式：F2

=SUMIF(A:A,E2,C:C)

说明：SUMIF函数的基本用法

	A	B	C	D	E	F	G
1	产品	单价	金额		产品	合计	
2	A	3	24		A	88	
3	B	4	12				
4	C	6	18				
5	A	1	8				
6	B	4	12				
7	A	2	16				
8	B	4	12				
9	B	4	12				
10	C	6	18				
11	A	2	16				
12	B	4	12				
13	A	3	24				

公式：

=SUMIF(A:A,E2,C:C)

3、单条件模糊求和

公式：详见下图

说明：如果需要进行模糊求和，就需要掌握通配符的使用，其中星号是表示任意多个字符，如"*A*"就表示a前和后任意多个字符，即包含A。

	A	B	C	D
1	产品	单价	金额	
2	ABC	3	24	
3	BD	4	12	
4	CA	6	18	
5				
6	1. 包含A的求和	42		
7	2. 以A开始求和	24		
8	3. 以A结束求和	18		
9				
10				
11	公式1=SUMIF(A2:A4,"*A*",C2:C4)			
12	公式2=SUMIF(A2:A4,"A*",C2:C4)			
13	公式3=SUMIF(A2:A4,"*A",C2:C4)			

4、多条件模糊求和

公式：C11

=SUMIFS(C2:C7,A2:A7,A11&"*",B2:B7,B11)

说明：在sumifs中可以使用通配符*

	A	B	C	D
1	产品	地区	数量	
2	电视21寸	郑州	2	
3	洗衣机5公斤	洛阳	5	
4	电视21寸	南宁	1	
5	洗衣机6公斤	北京	5	
6	电视21寸	郑州	5	
7	电视29寸	郑州	2	
8				
9				
10	产品	地区	数量	
11	电视	郑州	9	

5、多表相同位置求和

公式: b2

=SUM(Sheet1:Sheet19!B2)

说明: 在表中间删除或添加表后, 公式结果会自动更新。

Sheet3:B2			
	A	B	C
1	产品	本月累计	
2	A	19	
3	B	0	
4	C	0	
5	D	0	
6			

6、按日期和产品求和

公式: F2

=SUMPRODUCT((MONTH(\$A\$2:\$A\$25)=F\$1)*(\$B\$2:\$B\$25=\$E2)*\$C\$2:\$C\$25)

说明: SUMPRODUCT可以完成多条件求和

	A	B	C	D	E	F	G	H
1	日期	产品	销量	产品	1	2	3	
2	2015-1-9	A	5	A	23	50	26	
3	2015-1-10	A	18	B	16	20	0	
4	2015-1-11	B	11	C	0	35	20	
5	2015-1-12	B	5	D	0	16	1	
6	2015-2-10	A	18	S	0	11	0	
7	2015-2-11	C	19	E	0	20	0	
8	2015-2-15	D	6	F	0	2	0	
9	2015-2-15	C	15	G	0	10	0	
10	2015-2-15	S	11					
11	2015-2-18	A	16					

五、查找与引用公式

1、单条件查找公式

公式1: C11

=VLOOKUP(B11,B3:F7,4,FALSE)

说明: 查找是VLOOKUP最擅长的, 基本用法

2、双向查找公式

公式:

=INDEX(C3:H7,MATCH(B10,B3:B7,0),MATCH(C10,C2:H2,0))

说明: 利用MATCH函数查找位置, 用INDEX函数取值

	A	B	C	D	E	F	G	H
1								
2		姓名	1月	2月	3月	4月	5月	6月
3		刘名	10	54	2	54	5	57
4		吴号码	2	21	21	5	3	48
5		张晴	21	544	20	545	5	545
6		李栋	45	2	35	2	2	1
7		吴风	12	45	21	235	47	54
8								
9		姓名	月份	销售量				
10		张晴	3月	20				
11		李栋	4月					
12								

3、查找最后一条符合条件的记录。

公式: 详见下图

说明: 0/(条件)可以把不符合条件的变成错误值, 而lookup可以忽略错误值

	A	B	C	D	E
1					
2	例1: 查找A的最新单价				
3					
4		入库时间	产品名称	入库单价	
5		2012-1-2	A	10	
6		2012-1-3	B	34	
7		2012-1-4	A	19	
8		2012-1-5	E	25	
9		2012-1-6	A	12	
10		2012-1-7	C	25	
11					
13		A	12		
14					
15		C13=LOOKUP(1,0/(C5:C10=B13),D5:D10)			

4、多条件查找

公式:详见下图

说明:公式原理同上一个公式

	A	B	C	D	E	F	G
22	例2: 多条件查找						
23							
24		入库时间	产品名称	入库单价			
25		2012-1-2	A	10			
26		2012-1-3	B	34			
27		2012-1-4	A	19			
28		2012-1-5	E	25			
29		2012-1-6	A	12			
30		2012-1-7	C	25			
31							
33		入库时间	2012-1-4				
34		产品名称	A				
35		入库单价	19				
36							
37		C35=LOOKUP(1,0/((B25:B30=C33)*(C25:C30=C34)),D25:D30)					
38							

5、指定区域最后一个非空值查找

公式:详见下图

说明: 略

	A	B	C	D	E
1	例3: 查找最后一次还款日期				
2					
3	姓名	张越	李梅雨	胡秋伟	赵大志
4	1月	4			5
5	2月		5		
6	3月	34		4	
7	4月				
8	5月			6	6
9	6月	5	4		
10	7月				
11	8月				
12	9月			75	
13	10月	5			
14	11月				7
15	12月				
16	最后一次还款日期	10月	6月	9月	11月
17					
18	B14=LOOKUP(1, 0/(B2:B13<>""),\$A2:\$A13)				

6、按数字区间取对应的值

公式: 详见下图

公式说明: VLOOKUP和LOOKUP函数都可以按区间取值, 一定要注意, 销售量列的数字一定要升序排列。

A	B	C	D	E
1	例3：根据提成表计算销售提成			
2	提成表			
3	销售量	单个提成	B列表示区间	
4	0	5	0~49	
5	50	9	50~199	
6	200	15	>=200	
7				
8	姓名	销售数量	单个提成	提成额
9	张扬	2	5	10
10	李玉兰	300	15	4500
11	张千顺	15	5	75
12	吴柳	150	9	1350
13	孟良	100	9	900
14				
15	D9公式：=VLOOKUP(C9,B\$4:C\$6,2)			
16	用lookup =LOOKUP(C9,B\$4:B\$6,C\$4:C\$6)			
17	用IF 区间多时公式太复杂，不适用于回忆未来			

六、字符串处理公式

1、多单元格字符串合并

公式：c2

=PHONETIC(A2:A7)

说明：Phonetic函数只能对字符型内容合并，数字不可以。

	C2			
1	姓名			
2	D			
3	C			
4	R			
5	Q			
6	W			
7	B			
8				

2、截取除后3位之外的部分

公式：

=LEFT(D1,LEN(D1)-3)

说明：LEN计算出总长度,LEFT从左边截总长度-3个

3、截取-前的部分

公式:B2

=Left(A1,FIND("-",A1)-1)

说明：用FIND函数查找位置，用LEFT截取。

	A	B
1	科目	一级科目
2	银行存款-建行	银行存款
3	应收账款-张三	应收账款
4	其他应收款-赵志东	其他应收款
5		
6		

4、截取字符串中任一段的公式

公式:B1

=TRIM(MID(SUBSTITUTE(\$A1," ",REPT(" ",20)),20,20))

说明:公式是利用强插N个空字符的方式进行截取

	A	B	C	D	E	F
1	32 56 176 12 22	32	56	176	12	22
2						
3						

5、字符串查找

公式: B2

=IF(COUNT(FIND("河南",A2))=0,"否","是")

说明: FIND查找成功, 返回字符的位置, 否则返回错误值, 而COUNT可以统计出数字的个数, 这里可以用来判断查找是否成功。

	A	B
1	地址	是否河南
2	河南洛阳市	是
3	山东济南	否
4	河南南阳	是
5	河北石家庄	否
6		

6、字符串查找一对多

公式: B2

=IF(COUNT(FIND({"辽宁","黑龙江","吉林"},A2))=0,"其他","东北")

说明: 设置FIND第一个参数为**常量数组**, 用COUNT函数统计FIND查找结果

	A	B	C
1	地址	地区	
2	河南洛阳市	其他	
3	山东济南	其他	
4	辽宁铁岭	东北	
5	河南南阳	其他	
6	河北石家庄	其他	
7	黑龙江哈尔滨	东北	
8			

七、日期计算公式

1、两日期相隔的年、月、天数计算

A1是开始日期 (2011-12-1), B1是结束日期(2013-6-10)。计算:

相隔多少天? =DATEDIF(A1,B1,"d") 结果: 557

相隔多少月? =DATEDIF(A1,B1,"m") 结果: 18

相隔多少年? =DATEDIF(A1,B1,"Y") 结果: 1

不考虑年相隔多少月? =DATEDIF(A1,B1,"Ym") 结果: 6

不考虑年相隔多少天? =DATEDIF(A1,B1,"YD") 结果: 192

不考虑年月相隔多少天? =DATEDIF(A1,B1,"MD") 结果: 9

[DATEDIF函数](#)第3个参数说明:

"Y" 时间段中的整年数。

"M" 时间段中的整月数。

"D" 时间段中的天数。

"MD" 天数的差。忽略日期中的月和年。

"YM" 月数的差。忽略日期中的日和年。

"YD" 天数的差。忽略日期中的年。

2、扣除周末天数的工作日天数

公式: C2

=NETWORKDAYS.INTL(IF(B2<DATE(2015,1,1),DATE(2015,1,1),B2),DATE(2015,1,31),11)

说明: 返回两个日期之间的所有工作日数, 使用参数指示哪些天是周末, 以及有多少天是周末。周末和任何指定为假期的日期不被视为**工作日**

	A	B	C
1	姓名	入职日期	工作日天数
2	杨立	2014/2/1	27
3	吴天明	2015/1/10	19
4	张小燕	2013/8/14	27
5	刘真真	2015/1/6	23
6			

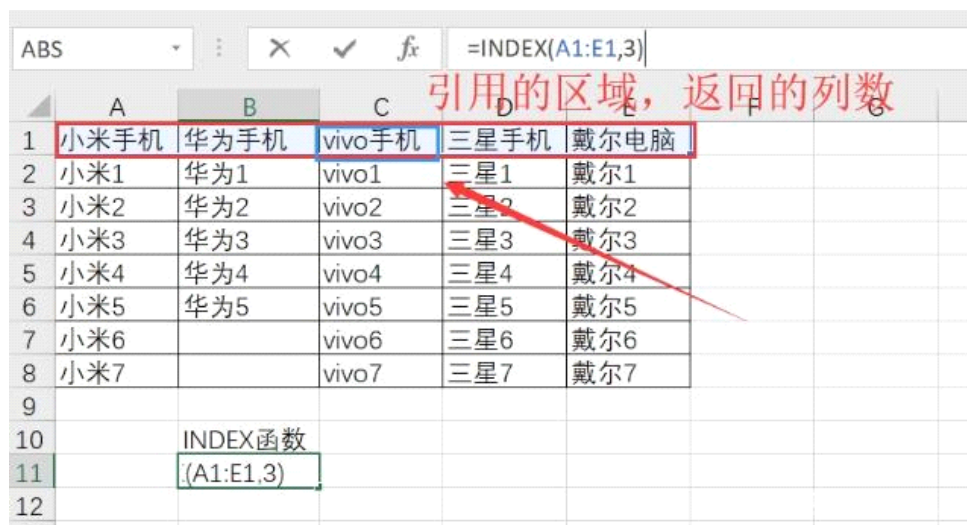
来自 <<https://zhuanlan.zhihu.com/p/94102393>>

3、INDEX函数（引用值或区域）

2021年12月17日 11:03

INDEX函数是返回表或区域中的值或值的引用。函数INDEX()有两种形式：[数组](#)形式和引用形式。数组形式通常返回数值或数值数组；引用形式通常返回引用。

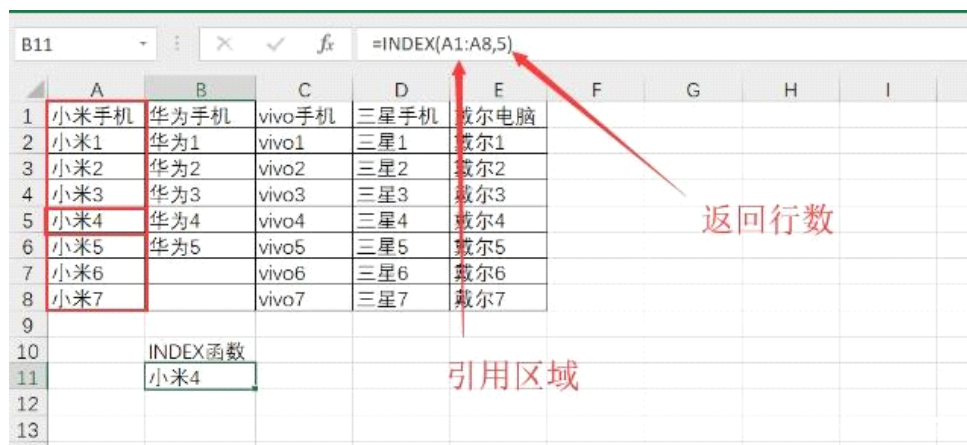
INDEX函数的语法=INDEX（区域或数组，返回列）



	A	B	C	D	E
1	小米手机	华为手机	vivo手机	三星手机	戴尔电脑
2	小米1	华为1	vivo1	三星1	戴尔1
3	小米2	华为2	vivo2	三星2	戴尔2
4	小米3	华为3	vivo3	三星3	戴尔3
5	小米4	华为4	vivo4	三星4	戴尔4
6	小米5	华为5	vivo5	三星5	戴尔5
7	小米6		vivo6	三星6	戴尔6
8	小米7		vivo7	三星7	戴尔7

INDEX函数
(A1:E1,3)

=INDEX（区域或数组，返回行）



	A	B	C	D	E
1	小米手机	华为手机	vivo手机	三星手机	戴尔电脑
2	小米1	华为1	vivo1	三星1	戴尔1
3	小米2	华为2	vivo2	三星2	戴尔2
4	小米3	华为3	vivo3	三星3	戴尔3
5	小米4	华为4	vivo4	三星4	戴尔4
6	小米5	华为5	vivo5	三星5	戴尔5
7	小米6		vivo6	三星6	戴尔6
8	小米7		vivo7	三星7	戴尔7

INDEX函数
小米4

=INDEX（区域或数组，返回行，返回列）

	A	B	C	D	E	F	G
1	费用项目	1月	2月	3月	4月	5月	6月
2	绿化费	6000	2000	2000	3000	6000	5000
3	办公费	5000	2000	3000	4000	5000	4000
4	宣传费	2000	3000	4000	8000	5000	1000
5	工资	3000	4000	2000	4000	3000	2000
6	福利费	3000	2000	3000	5000	4000	2000
7							
8							
9	月份	费用项目	费用额				
10	3月	工资	2000				
11							

分析：

先用MATCH函数查找3月在第一行中的位置

=MATCH(B10,\$A\$2:\$A\$6,0)

再用MATCH函数查找费用项目在A列的位置

= MATCH(A10,\$B\$1:\$G\$1,0)

最后用INDEX根据行数和列数提取数值

INDEX(区域,行数,列数)

=INDEX(B2:G6,MATCH(B10,\$A\$2:\$A\$6,0),MATCH(A10,\$B\$1:\$G\$1,0))